

GPUMODE Lecture Study Notes: Lecture 104: Gluon and Linear Layouts

Core Problem the Lecture Addresses

This lecture explains why modern GPU kernel DSLs need to expose more hardware structure than classical Triton-style programming, while still preserving tensor-level productivity. The main technical problem is:

How can a programmer write speed-of-light kernels for modern GPUs, especially Blackwell, without manually writing every low-level CUDA, PTX, shared-memory, barrier, and tensor-core detail?

The lecture presents **Gluon** as a lower-level kernel DSL than Triton, designed to expose explicit hardware concepts such as layouts, warp specialization, shared memory, asynchronous tensor memory access, tensor-core instructions, and memory barriers. It then explains **linear layouts**, the mathematical framework used to represent data placement across registers, lanes, warps, shared memory, tensor memory, and logical tensor indices.

The key bottleneck is that the hardware execution model has diverged from the high-level Triton programming model. On older GPUs such as Ampere, a typical Triton program matched the hardware reasonably well:

- A group of warps cooperatively loaded input tiles.
- The same group synchronized.
- The same group prepared data for MMA instructions.
- The same group executed tensor-core work.
- Control flow was mostly lockstep across all warps.

On Blackwell, high-performance kernels are structurally different:

- Some warps may only issue TMA loads.
- Some warps may only issue tensor-core MMA instructions.
- Other warps may only perform epilogue computation and stores.

- Tensor memory, shared memory, and asynchronous barriers introduce extra execution channels.
- The optimal schedule is not a single lockstep program but a pipeline of specialized warp partitions.

Thus, the main design tension is:

Productivity versus Direct hardware control.

Triton chooses a higher-level abstraction. CUDA and CuTe-style programming expose more detail. Gluon tries to sit between them:

Triton $\longrightarrow$ Gluon $\longrightarrow$ CUDA/PTX/CuTe-like direct hardware programming.

Gluon still gives the programmer tensor abstractions, but requires explicit control over layouts and warp-specialized execution.

Required Background

GPU Execution Model

A GPU kernel is executed by a grid of thread blocks. In NVIDIA terminology, a thread block is also called a **CTA**, or cooperative thread array. Each CTA is scheduled onto an SM.

- **Thread**: scalar execution unit in the CUDA programming model.
- **Warp**: group of usually 32 NVIDIA threads executing in SIMD-like fashion.
- **CTA**: a thread block containing one or more warps.
- **SM**: streaming multiprocessor, the hardware unit that schedules warps and owns registers, shared memory, warp schedulers, and tensor-core pipelines.
- **CGA**: cooperative group array or cluster-level grouping of CTAs on newer hardware.

The effective performance of a kernel depends on how work is mapped across these hierarchy levels:

Logical tensor element $\longleftrightarrow$ register $\longleftrightarrow$ thread/lane $\longleftrightarrow$ warp $\longleftrightarrow$ CTA $\longleftrightarrow$ SM.

A layout is precisely the rule that defines this mapping.

Memory Hierarchy

The lecture repeatedly references the GPU memory hierarchy:

- **Global memory:** high-capacity HBM, large latency, high bandwidth.
- **L2 cache:** shared across SMs.
- **Shared memory:** programmer-controlled memory local to a CTA or CTA cluster.
- **Registers:** fastest per-thread storage.
- **Tensor memory:** Blackwell-specific accumulator storage used by TCGEN05 tensor-core operations.

A simplified performance model is:

$$\text{Runtime} \approx \max \left(\frac{\text{FLOPs}}{\text{Peak compute throughput}}, \frac{\text{Bytes moved}}{\text{Effective memory bandwidth}} \right).$$

A kernel is compute-bound when the first term dominates and memory-bound when the second term dominates.

Tensor Cores and MMA

Matrix multiplication kernels use tensor-core instructions. At a high level:

$$C = AB,$$

where a CTA computes a tile $C_{m,n}$ by iterating over K -tiles:

$$C_{m,n} = \sum_k A_{m,k} B_{k,n}.$$

A tensor-core MMA instruction computes a small matrix multiply-accumulate:

$$D = AB + C.$$

The exact fragment layouts required by the hardware are not arbitrary. The compiler or programmer must arrange values in registers, shared memory, or tensor memory in exactly the form expected by the instruction.

TMA and Asynchronous Execution

TMA, or tensor memory accelerator, is a hardware feature on Hopper and Blackwell that can move large tensor tiles asynchronously. Instead of every warp cooperatively issuing many scalar or vector loads, one warp can issue a TMA operation that transfers a large multidimensional tile.

The important consequence is that TMA does not behave like ordinary per-thread global loads:

ordinary load : warp executes load and receives value,

whereas

TMA load : warp issues request $\longrightarrow$ hardware transfer proceeds asynchronously.

Because TMA is asynchronous, programmers need barriers, phases, counters, and explicit producer-consumer synchronization.

Main Concepts

Why Triton Was Powerful

Triton became popular because it provided a productive tensor-level programming model while still enabling high performance. A programmer could write something close to the mathematical kernel and rely on the compiler to lower operations such as `tl.dot` into efficient tensor-core code.

A simplified Triton-style matmul looks like:

```
@triton.jit
def matmul_kernel(A, B, C, M: tl.constexpr, N: tl.constexpr, K: tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)

    acc = tl.zeros((BLOCK_M, BLOCK_N), tl.float32)

    for k0 in range(0, K, BLOCK_K):
        a = tl.load(A + offs_m[:, None] * K + (k0 + offs_k[None, :]))
        b = tl.load(B + (k0 + offs_k[:, None]) * N + offs_n[None, :])
        acc += tl.dot(a, b)
```

```
tl.store(C + offs_m[:, None] * N + offs_n[None, :], acc)
```

The important abstraction is `tl.dot`. It hides:

- register fragment layouts,
- shared memory staging,
- tensor-core instruction selection,
- pipelining,
- vectorization,
- warp scheduling details.

This worked very well when the hardware execution structure resembled the program structure. On Ampere, the same cooperating warps performed load, synchronization, MMA, and loop iteration.

Why Triton Becomes Harder on Blackwell

Blackwell introduces a more heterogeneous execution style inside one CTA. Different warp groups may have different roles:

CTA warps = TMA producer warps $\cup$ MMA consumer warps $\cup$ epilogue warps $\cup$ scheduling warps.

This is not naturally represented as one lockstep tensor program. The compiler must infer a major restructuring:

single high-level loop $\longrightarrow$ multiple asynchronous warp-specialized pipelines.

The lecture argues that this becomes brittle for complex production kernels such as attention, convolutions, and fused kernels. A compiler can pattern-match simple matmul, but arbitrary kernels require choices about:

- which warps produce data,
- which warps consume data,
- where shared memory buffers live,
- how many pipeline stages to use,
- where barriers and fences are required,
- how much register budget each warp partition receives,
- when to convert layouts,

- how to avoid bank conflicts,
- how to overlap TMA, MMA, and epilogue work.

This motivates Gluon.

Gluon’s Position

Gluon is designed to be lower-level than Triton but higher-level than raw CUDA or PTX.

Triton	:	high-level tensor DSL, compiler controls many hardware choices,
Gluon	:	explicit layouts, warp specialization, tensor abstractions,
CUDA/PTX	:	manual hardware programming with thread-level details.

The goal is not automatic portability. Instead, Gluon prioritizes enabling expert programmers to write near speed-of-light kernels for specific hardware generations.

Layouts as First-Class Objects

The defining idea in Gluon is that tensors are not just shaped arrays. They also have explicit layouts.

A logical tensor element is addressed by coordinates:

$$(i_0, i_1, \dots, i_{d-1}).$$

A hardware placement is described by coordinates such as:

$$(r, t, w),$$

where:

- r is a register coordinate,
- t is a lane or thread coordinate inside a warp,
- w is a warp coordinate inside a CTA.

A layout maps hardware coordinates to logical coordinates:

$$L : (r, t, w) \mapsto (i_0, i_1, \dots, i_{d-1}).$$

This mapping determines:

- which thread owns which tensor element,

- whether global loads are coalesced,
- whether reductions can be implemented with warp shuffles,
- whether shared memory access has bank conflicts,
- whether tensor-core operands are in a legal layout,
- how expensive layout conversion is.

Common Gluon Layout Types

The lecture mentions several layout classes.

Blocked Layout

A blocked layout distributes a tensor tile across registers, lanes, and warps in a structured way. For example, a 16×16 matrix might be distributed across:

$$2 \text{ warps} \times 32 \text{ lanes per warp} \times 4 \text{ registers per lane} = 256 \text{ elements.}$$

This exactly covers:

$$16 \times 16 = 256.$$

A blocked layout might specify:

- elements per thread,
- threads per warp in each logical dimension,
- warps per CTA in each logical dimension,
- dimension order, such as row-major or column-major distribution.

Slice Layout

A slice layout is derived when reducing or removing a dimension.

If a tensor has shape:

$$X \in \mathbb{R}^{M \times N}$$

and we reduce over dimension 0, the result has shape:

$$Y_j = \sum_{i=0}^{M-1} X_{i,j}, \quad Y \in \mathbb{R}^N.$$

The resulting layout removes the reduced dimension. In the linear layout framework, this corresponds to zeroing or projecting out basis contributions for that dimension.

Auto Layout

An auto layout defers layout selection. The compiler propagates layout constraints backward from explicitly specified uses.

The important design rule is:

Gluon does not invent arbitrary new layouts. It propagates and transforms layouts derived from choices specified in the program.

Coalesced Layout

A coalesced layout chooses a layout suitable for efficient loads and stores. It considers pointer alignment and continuity to choose vectorized memory operations.

The goal is to make adjacent lanes access adjacent memory addresses:

$$\text{lane } t \mapsto \text{address } a + t \cdot s,$$

with s chosen so the memory transaction is coalesced and vectorizable.

Explicit Warp Specialization

Gluon’s major feature is explicit warp specialization. A CTA can be partitioned into groups of warps, where each group has a different role.

For example, a Blackwell matmul may use:

TMA partition	:	issues asynchronous global-to-shared loads,
MMA partition	:	issues tensor-core operations,
Epilogue partition	:	loads accumulators and stores output,
CLC partition	:	performs cluster launch control scheduling.

A warp partition is therefore a group of warps assigned a role.

$$\text{CTA} = P_{\text{load}} \cup P_{\text{mma}} \cup P_{\text{epi}} \cup P_{\text{schedule}}.$$

Gluon’s warp specialization follows a fork-join model. The ordinary program reaches a specialization point, then control splits into multiple workers. When all workers finish, control rejoins. In many optimized kernels, the useful work ends inside the specialized region.

Shared Memory as a Tensor Abstraction

Gluon exposes shared memory directly, but still treats it as a tensor. A programmer allocates shared memory with a shape and layout. For pipelining, the first dimension may be a non-power-of-two stage dimension:

$$\text{SMEM shape} = (S, M, N),$$

where S is the number of pipeline stages, possibly 3, and the remaining logical dimensions are usually powers of two.

The access pattern is tensor-like:

store tensor tile into SMEM stage s ,

then later:

load tensor tile from SMEM stage s .

The compiler lowers this into per-thread shared-memory stores and loads, inserts synchronization where visible, and may insert fences for separate hardware access paths.

Memory Barriers and Pipeline Counters

Asynchronous TMA and MMA operations require barriers. A typical pipeline stage uses:

- an index identifying the current stage,
- a phase bit toggling between 0 and 1,
- a memory barrier tracking producer-consumer completion.

A simplified immutable barrier counter abstraction is:

```
@dataclass(frozen=True)
class BarrierCounter:
    index: int
    phase: int
    num_stages: int

    def advance(self):
        next_index = self.index + 1
        next_phase = self.phase

        if next_index == self.num_stages:
            next_index = 0
```

```

next_phase = 1 - self.phase

return BarrierCounter(next_index, next_phase, self.num_stages)

```

The key point is that Gluon values are immutable. Updating a state-like object returns a new object instead of modifying it in place. This matches SSA-style compiler representation and avoids accidentally lowering mutable values to stack memory, which would be catastrophic on a GPU.

Hardware-Level Explanation

Ampere-Style Cooperative Execution

On Ampere-like tensor-core kernels, a typical CTA-level matmul pipeline can be conceptualized as:

load $\rightarrow$ sync $\rightarrow$ MMA $\rightarrow$ repeat $\rightarrow$ store.

All warps in the CTA usually follow the same loop and participate in each phase.

The Triton program structure and the hardware schedule are fairly aligned: every warp mostly does the same kind of work at the same time.

Blackwell-Style Specialized Execution

Blackwell enables a different pipeline:

TMA load $\parallel$ MMA $\parallel$ epilogue store $\parallel$ work scheduling.

The important hardware features are:

- **TMA**: one warp can issue large asynchronous tensor tile loads.
- **TCGEN05.MMA**: one warp can issue large tensor-core operations.
- **Tensor memory**: accumulators may live in tensor memory instead of ordinary registers.
- **M barriers**: shared-memory-resident barriers used for asynchronous completion tracking.
- **Cluster-level features**: CTAs in a cluster can share or multicast data.

This creates a producer-consumer pipeline:

$P_{\text{load}} \xrightarrow{\text{TMA}} \text{shared memory} \xrightarrow{\text{MMA}} \text{tensor memory} \xrightarrow{\text{load}} \text{registers} \xrightarrow{\text{store}} \text{global memory}.$

Why Layout Controls Lowering

A layout determines where every logical element lives. Therefore it controls the lowering of tensor operations.

For example, suppose a reduction computes:

$$y_j = \sum_i x_{i,j}.$$

If the i -dimension is local to a warp, then the reduction can be implemented with warp shuffles:

$$x_t \leftarrow x_t + x_{t \oplus 1}, \quad x_t \leftarrow x_t + x_{t \oplus 2}, \quad x_t \leftarrow x_t + x_{t \oplus 4}, \quad \dots$$

If the reduction dimension spans multiple warps, shared memory and CTA-level synchronization may be required.

Thus:

$$\text{layout choice} \implies \text{communication mechanism}.$$

Register Budgeting

In warp-specialized kernels, different partitions need different register budgets. For example:

$$\begin{aligned} P_{\text{TMA}} &: \text{few registers,} \\ P_{\text{MMA}} &: \text{many registers or tensor-memory control state,} \\ P_{\text{epilogue}} &: \text{registers for output fragments, scaling, activation, stores.} \end{aligned}$$

The lecture emphasizes that register allocation is difficult for the compiler to predict because final allocation decisions are made deep in the compiler stack by tools such as `ptxas`. Too few registers cause spilling. Too many may reduce occupancy or cause compilation failure.

A rough occupancy constraint is:

$$\text{active CTAs per SM} \leq \left\lfloor \frac{R_{\text{SM}}}{R_{\text{thread}} \cdot T_{\text{CTA}}} \right\rfloor,$$

where:

- R_{SM} is the register file capacity per SM,
- R_{thread} is registers per thread,

- T_{CTA} is threads per CTA.

However, in warp-specialized programs, this model is complicated because different warp partitions may have different register needs.

Shared Memory Channels and Fences

On Hopper and Blackwell, TMA and MMA may access shared memory through different internal channels than ordinary loads and stores. Therefore some producer-consumer paths require fences.

A conceptual dependency is:

$$\text{ordinary store to SMEM} \rightarrow \text{TMA or MMA reads SMEM.}$$

If these use different hardware channels, a fence may be needed to make writes visible. Gluon can infer common cases, but if data crosses warp-specialized partitions in ways the compiler cannot see, the programmer may need to insert a manual fence.

Visual Concept

Draw one CTA as a rectangle split into four warp partitions: TMA load warp, MMA warp, epilogue warps, and CLC scheduling warp. Show arrows from global memory to shared memory, shared memory to tensor cores, tensor memory to registers, and registers or shared memory back to global memory.

Mathematical Reasoning

Layouts as Functions

A layout can be described as a function:

$$L : \mathcal{H} \rightarrow \mathcal{T},$$

where:

- $\mathcal{H}$ is the hardware coordinate space,
- $\mathcal{T}$ is the logical tensor coordinate space.

For a simple distributed tile:

$$\mathcal{H} = \{(r, t, w)\},$$

where r is register, t is thread lane, and w is warp. For a two-dimensional tensor:

$$\mathcal{T} = \{(i, j)\}.$$

Thus:

$$L(r, t, w) = (i, j).$$

Example: A 16×16 Tile

Suppose a 16×16 matrix tile is distributed across:

$$4 \text{ registers per thread} \times 32 \text{ lanes per warp} \times 2 \text{ warps} = 256$$

hardware slots, equal to:

$$16 \times 16 = 256.$$

One possible block layout can be described using integer formulas:

$$i = \left\lfloor \frac{r}{2} \right\rfloor + 2 \left\lfloor \frac{t}{8} \right\rfloor + 8w,$$

$$j = (r \bmod 2) + 2(t \bmod 8).$$

Here:

- $r \in \{0, 1, 2, 3\},$
- $t \in \{0, \dots, 31\},$
- $w \in \{0, 1\},$
- $i, j \in \{0, \dots, 15\}.$

The terms explain the layout:

$$\left\lfloor \frac{r}{2} \right\rfloor$$

moves in the row dimension every two registers, while

$$r \bmod 2$$

moves in the column dimension inside a thread.

Similarly,

$$t \bmod 8$$

uses the lower three bits of the lane ID to move across columns, while

$$\left\lfloor \frac{t}{8} \right\rfloor$$

uses higher lane bits to move down rows.

Power-of-Two Structure

Most efficient GPU tiling dimensions are powers of two. This allows the layout formulas to be interpreted as bit manipulation.

Let:

$$r = (r_1 r_0)_2,$$

$$t = (t_4 t_3 t_2 t_1 t_0)_2,$$

$$w = (w_0)_2.$$

Then the previous layout can be viewed as bit placement:

$$i = (w_0, t_4, t_3, r_1)_2,$$

$$j = (t_2, t_1, t_0, r_0)_2.$$

This is the conceptual jump toward linear layouts: layouts become bit transformations.

Linear Layout Definition

A **linear layout** is a linear map over the finite field $\mathbb{F}_2$:

$$L : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m.$$

Because this is a finite-dimensional linear map, it can be represented by a binary matrix:

$$A \in \mathbb{F}_2^{m \times n}.$$

The layout maps input bits x to output bits y :

$$y = Ax.$$

Arithmetic is over $\mathbb{F}_2$, meaning:

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = 0.$$

Thus addition is XOR:

$$a + b \equiv a \oplus b.$$

Multiplication is AND:

$$ab \equiv a \wedge b.$$

Therefore each output bit is:

$$y_i = \bigoplus_{j=0}^{n-1} A_{ij} \wedge x_j.$$

Permutation Layouts as Matrices

If the layout only rearranges bits, then A is a permutation matrix. Each row and column has exactly one 1.

For example, if:

$$y_0 = x_2, \quad y_1 = x_0, \quad y_2 = x_1,$$

then:

$$A = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

A permutation matrix captures layouts similar to pure striding or reshaping, but linear layouts are more general because rows may contain multiple 1 values. Multiple 1 values create XOR combinations, which model swizzling.

Swizzling as Linear Algebra

Swizzling is used to reduce shared-memory bank conflicts. The lecture shows that swizzling can be expressed naturally as a linear layout.

Suppose shared-memory offset bits are:

$$o = (o_5, o_4, o_3, o_2, o_1, o_0)_2.$$

A non-swizzled mapping might map these bits directly to logical coordinates. A swizzled mapping can define an output bit as an XOR:

$$y_0 = o_0 \oplus o_3.$$

In matrix form, this row has two 1 entries:

$$\begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

Thus swizzling is not an extra ad-hoc rule. It is just another linear map over $\mathbb{F}_2$.

Broadcasting and Zero Bases

A zero basis column means that changing a hardware coordinate does not change the logical coordinate.

If column j of A is all zeros:

$$Ae_j = 0,$$

then the layout does not depend on input bit x_j . Multiple hardware positions map to the same logical element. This corresponds to broadcasting or reduction ownership.

For reduction, this is useful because many lanes may own partial contributions to the same output element. A reduction can then combine those values with a shuffle or shared-memory reduction.

Layout Conversion

Suppose a tensor is currently in distributed register layout A :

$$A : \mathcal{H}_{\text{register}} \rightarrow \mathcal{T}.$$

Suppose we want to store it into shared-memory layout B :

$$B : \mathcal{H}_{\text{shared}} \rightarrow \mathcal{T}.$$

To compute where a register-owned logical element should be stored in shared memory, we need:

$$\mathcal{H}_{\text{register}} \xrightarrow{A} \mathcal{T} \xrightarrow{B^{-1}} \mathcal{H}_{\text{shared}}.$$

Thus the conversion map is:

$$C = B^{-1}A.$$

This turns layout conversion into linear algebra. Instead of writing hand-coded indexing rules for every pair of layouts, the compiler solves a matrix problem.

Why This Removes Bugs

Before linear layouts, compilers needed many special cases:

blocked $\rightarrow$ MMA layout,

MMA layout $\rightarrow$ shared layout,

slice layout $\rightarrow$ blocked layout,

shared layout $\rightarrow$ tensor memory layout.

Each conversion required hand-written offset arithmetic. Bugs could cause compile failures, runtime crashes, or silently wrong results. With linear layouts, conversions are computed systematically:

$$\text{layout problem} \longrightarrow \text{matrix problem over } \mathbb{F}_2.$$

Bank Conflict Reasoning

Shared memory is divided into banks. If multiple lanes access the same bank in the same transaction, a bank conflict may occur.

Let:

$$b(a)$$

be the bank selected by shared-memory address a . A conflict occurs when distinct lanes $t_1 \neq t_2$ access addresses with:

$$b(a_{t_1}) = b(a_{t_2}).$$

A good swizzle changes the address mapping:

$$a_t = S(t, r, w),$$

so that bank indices are distributed across lanes:

$$b(a_{t_1}) \neq b(a_{t_2}) \quad \text{for as many active lanes as possible.}$$

Linear layouts allow the compiler to search for or derive swizzles mathematically, while also preserving vectorization.

Arithmetic Intensity

For matrix multiplication:

$$C = AB,$$

where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the FLOP count is approximately:

$$\text{FLOPs} = 2MNK.$$

If data is loaded naively once:

$$\text{Bytes} = s(MK + KN + MN),$$

where s is bytes per element. The arithmetic intensity is:

$$\text{AI} = \frac{2MNK}{s(MK + KN + MN)}.$$

Tiling increases effective arithmetic intensity by reusing A and B tiles from shared memory or tensor memory before returning to global memory.

Code Walkthrough

Gluon-Style Layout-Aware Kernel Structure

A Gluon kernel resembles Triton but has explicit layout arguments. A conceptual sketch is:

```
@gluon.jit
def kernel(x_ptr, y_ptr, n: gl.constexpr, layout: gl.constexpr):
    pid = gl.program_id(0)

    offsets = pid * BLOCK + gl.arange(0, BLOCK, layout=layout)

    x = gl.load(x_ptr + offsets, layout=layout)
    y = x * x + 1.0

    gl.store(y_ptr + offsets, y, layout=layout)
```

The key difference from Triton is that the layout is not implicit. The `layout` object controls how the abstract tensor `offsets` is distributed across hardware.

Major line explanations:

- `gl.arange(..., layout=layout)` creates a tile whose elements are owned according to the specified layout.
- `gl.load` uses the layout to determine which lanes issue which memory transactions.
- `gl.store` similarly maps tensor elements back to memory addresses.
- A coalesced layout can make adjacent lanes issue adjacent memory accesses.

Representative Warp Specialization Sketch

A conceptual Gluon-style warp-specialized kernel looks like:

```
@gluon.jit
def matmul_kernel(A, B, C, desc):
    def load_partition():
        for stage in gl.static_range(NUM_STAGES):
            issue_tma_load(A, B, stage)
            arrive_tma_barrier(stage)

    def mma_partition():
        for k_tile in gl.static_range(NUM_K_TILES):
            wait_tma_barrier(k_tile)
            issue_tcgen05_mma(k_tile)
            arrive_mma_barrier(k_tile)
```

```

def epilogue_partition():
    wait_mma_barrier()
    acc = load_tensor_memory_to_registers()
    out = apply_epilogue(acc)
    store_output(C, out)

gl.warp_specialize(
    load_partition,
    mma_partition,
    epilogue_partition
)

```

This code expresses what Triton would have to infer: separate producer, compute, and epilogue roles.

The hardware behavior is:

- The load partition issues asynchronous TMA transfers.
- The MMA partition waits for TMA completion and issues tensor-core work.
- The epilogue partition waits for accumulation completion.
- The three partitions overlap in time across pipeline stages.

Shared Memory Allocation

A conceptual shared-memory allocation is:

```

smem_a = gl.allocate_shared(
    shape=(NUM_STAGES, BLOCK_M, BLOCK_K),
    dtype=gl.float16,
    layout=shared_layout_a
)

smem_b = gl.allocate_shared(
    shape=(NUM_STAGES, BLOCK_K, BLOCK_N),
    dtype=gl.float16,
    layout=shared_layout_b
)

```

The first dimension is the pipeline stage. If:

$$\text{NUM_STAGES} = 3,$$

then producer and consumer partitions can operate on different stages:

$$\text{load stage } s + 1 \quad || \quad \text{MMA stage } s \quad || \quad \text{epilogue stage } s - 1.$$

The compiler statically packs shared memory allocations to reduce total usage.

Asynchronous TMA Load Sketch

A simplified form of a TMA load sequence is:

```
barrier = gl.allocate_mbarrier(num_stages=NUM_STAGES)

for stage in gl.static_range(NUM_STAGES):
    barrier.prepare(stage, expected_bytes=BYTES_A + BYTES_B

    gl.nv.tma_async_load(
        src=A_global_tile,
        dst=smem_a[stage],
        barrier=barrier[stage]
    )

    gl.nv.tma_async_load(
        src=B_global_tile,
        dst=smem_b[stage],
        barrier=barrier[stage]
    )
```

The important hardware effects are:

- TMA operations are asynchronous.
- The barrier counts expected transferred bytes.
- Consumer partitions must wait before reading shared memory.
- A single warp may issue a large transfer.

MMA Partition Sketch

```
for k_tile in gl.static_range(NUM_K_TILES):
    stage = k_tile % NUM_STAGES

    barrier.wait(stage)

    gl.nv.tcgen05_mma(
        a=smem_a[stage],
        b=smem_b[stage],
        acc=tensor_memory_accumulator,
        layout_a=mma_shared_layout_a,
        layout_b=mma_shared_layout_b,
        layout_acc=tensor_memory_layout
    )
```

```
mma_barrier.arrive(stage)
```

The key point is that `TCGEN05.MMA` consumes operands from shared memory or tensor memory and writes accumulators into tensor memory. This differs from older tensor-core paths where accumulators are ordinary register fragments.

Epilogue Sketch

```
mma_barrier.wait(final_stage)
```

```
acc_regs = gl.nv.tcg05_ld(
    tensor_memory_accumulator,
    layout=epilogue_layout
)
```

```
out = acc_regs.to(gl.float32)
```

```
gl.store(C + output_offsets, out, layout=epilogue_layout)
```

The epilogue may include:

- type conversion,
- scaling,
- bias addition,
- activation,
- quantization,
- vectorized global stores.

For high-bandwidth stores, a kernel may route register data through shared memory and then use an asynchronous store path.

Two-CTA Mode

Blackwell supports a two-CTA mode for certain MMA patterns. Without two-CTA mode, two CTAs might replicate the same B tile:

$$B_{\text{CTA } 0} = B_{\text{tile}}, \quad B_{\text{CTA } 1} = B_{\text{tile}}.$$

With two-CTA mode, the B tile can be split across CTAs in the same cluster:

$$B_{\text{tile}} = B_{\text{CTA } 0} \cup B_{\text{CTA } 1}.$$

This reduces shared-memory usage and allows larger N -dimension tiles.

A conceptual setup is:

```
tm_layout = gl.nv.mma.tensor_memory_layout(  
    shape=(TMEM_M, TMEM_N),  
    cta_layout=cta_layout,  
    two_cta=True  
)  
  
shared_layout_b = gl.nv.mma.shared_layout(  
    block_shape=(BLOCK_K, BLOCK_N),  
    dtype=gl.float16,  
    cta_layout=cta_layout,  
    two_cta=True  
)
```

The compiler verifies that the layout is compatible with the required two-CTA instruction mode.

TMA Multicast

TMA multicast allows one CTA to issue a TMA load and replicate data into multiple CTA shared-memory regions in a cluster.

Without multicast:

CTA 0 loads tile, CTA 1 loads same tile, CTA 2 loads same tile, CTA 3 loads same tile.

With multicast:

CTA 0 issues one load $\longrightarrow$ tile appears for CTA 0, CTA 1, CTA 2, CTA 3.

This reduces L2 and HBM traffic.

A simplified layout idea for a four-CTA cluster is:

$$\text{LHS CTA layout bases} = \{(1, 0), (2, 0)\},$$

meaning CTAs split the M -dimension.

For the RHS tile:

$$\text{RHS CTA layout bases} = \{(0, 1), (0, 0)\}.$$

The first basis splits the N -dimension across two CTAs. The second basis repeats data across the other CTA group, triggering multicast behavior.

Cluster Launch Control

Cluster launch control, or CLC, is a Blackwell feature for dynamically reassigning future work to already resident SMs. It improves load balancing when static tile assignment leaves some SMs idle.

A conceptual CLC worker does:

```
def clc_partition():
    while has_more_work():
        wait_for_compute_group_arrival()

        if can_cancel_future_tile():
            next_pid = cancel_future_tile()
            publish_next_pid_to_compute_groups(next_pid)
```

The purpose is:

idle or finishing SM $\longrightarrow$ steal future tile $\longrightarrow$ keep SM occupied.

This is especially valuable when tile runtimes are imbalanced.

Performance Intuition

Why Explicit Layouts Matter

Explicit layouts let the programmer control communication locality.

A reduction over a dimension can be cheap or expensive depending on layout:

same warp $\Rightarrow$ shuffle reduction,

same CTA but different warps $\Rightarrow$ shared-memory reduction,

different CTAs $\Rightarrow$ global synchronization or separate kernel.

Thus, a layout is not cosmetic. It decides whether an operation is register-local, warp-local, CTA-local, or global.

Why Convert Layout Can Be Dangerous

In high-level compilers, layout conversions may be inserted automatically. If a conversion remains inside a hot loop, the performance cost can dominate.

Suppose a loop performs:

load $\rightarrow$ convert layout $\rightarrow$ MMA

for every K -tile. If convert layout requires shared-memory traffic, synchronization, or bank-conflict-prone movement, the kernel may lose most of its tensor-core throughput.

Gluon makes conversions explicit enough that the programmer can decide:

convert once versus convert every iteration versus recompute in desired layout.

Why Linear Layouts Improve Compiler Reliability

Before linear layouts, each operation pair required custom reasoning. This creates a combinatorial explosion:

$$N \text{ layouts} \implies O(N^2) \text{ possible conversions.}$$

With linear layouts, the compiler has a common representation. The conversion problem becomes:

$$C = B^{-1}A.$$

This improves:

- correctness,
- extensibility,
- support for new hardware instructions,
- automatic swizzle generation,
- generic lowering of reductions and layout conversions.

Why Blackwell Matmul Needs Many Overlapped Stages

A speed-of-light Blackwell matmul overlaps:

TMA load || TCGEN05 MMA || tensor-memory load || epilogue || CLC scheduling.

The goal is to keep all hardware engines busy:

$$\eta_{\text{total}} \approx \min(\eta_{\text{TMA}}, \eta_{\text{MMA}}, \eta_{\text{epilogue}}, \eta_{\text{scheduling}}).$$

If any stage becomes the bottleneck, the whole pipeline slows.

Shared Memory Reuse

For a CTA tile:

$$A_{\text{tile}} \in \mathbb{R}^{B_M \times B_K}, \quad B_{\text{tile}} \in \mathbb{R}^{B_K \times B_N},$$

the number of multiply-adds is:

$$2B_M B_N B_K.$$

The loaded bytes are roughly:

$$s(B_M B_K + B_K B_N).$$

The tile arithmetic intensity is:

$$\text{AI}_{\text{tile}} = \frac{2B_M B_N B_K}{s(B_M B_K + B_K B_N)}.$$

Larger B_M and B_N increase reuse, but also increase:

- shared-memory usage,
- register or tensor-memory pressure,
- epilogue cost,
- scheduling complexity.

TMA Multicast and L2 Traffic

If q CTAs need the same tile, loading independently costs:

$$q \cdot \text{Bytes}_{\text{tile}}.$$

With multicast, the L2/HBM traffic can approach:

$$1 \cdot \text{Bytes}_{\text{tile}}.$$

The ideal reduction factor is:

$$\text{traffic reduction} = \frac{q \cdot \text{Bytes}_{\text{tile}}}{\text{Bytes}_{\text{tile}}} = q.$$

In practice, the benefit depends on cluster structure, alignment, tile shape, and whether the tile is truly reused across CTAs.

Two-CTA Mode and Shared Memory Footprint

If each CTA replicates B , the shared memory cost is:

$$2 \cdot sB_K B_N.$$

If two CTAs split B , the per-cluster cost can become:

$$sB_K B_N.$$

This can allow larger B_N or more pipeline stages under the same shared-memory budget.

Common Misconceptions

Misconception 1: A Smarter Compiler Can Always Recover the Optimal Schedule

A compiler can do a lot, but Blackwell kernels require many hardware-specific choices:

- warp partitioning,
- register budgeting,
- TMA staging,
- MMA staging,
- barrier placement,
- swizzle selection,
- shared-memory packing,
- tensor-memory layout,
- cluster-level work scheduling.

The lecture argues that fully inferring all these choices for arbitrary production kernels is brittle. Gluon exposes enough control for expert programmers to express the intended schedule directly.

Misconception 2: Layouts Are Just Tensor Strides

Strides describe how logical tensor coordinates map to linear memory addresses. Gluon layouts describe how logical tensor elements map to hardware resources such as registers, lanes, warps, shared-memory offsets, and tensor memory.

A PyTorch-style stride map is:

$$(i, j) \mapsto i \cdot s_i + j \cdot s_j.$$

A Gluon-style distributed layout is more like:

$$(r, t, w) \mapsto (i, j).$$

Linear layouts further express the mapping at the bit level over $\mathbb{F}_2$.

Misconception 3: Swizzling Is an Ad-Hoc Trick

Swizzling often appears as mysterious indexing code. Linear layouts show that swizzling is a linear transformation over bits:

$$y = Ax,$$

where some output bits are XORs of input bits. This makes swizzling composable, analyzable, and optimizable.

Misconception 4: Exposing Hardware Means the Compiler Does Nothing

Even though Gluon exposes hardware primitives, the compiler still performs major work:

- vectorizing loads and stores,
- checking alignment,
- lowering layout conversions,
- choosing shared-memory swizzles,
- minimizing bank conflicts,
- inserting synchronization,
- packing shared-memory allocations,
- splitting large TMA operations into legal hardware-sized pieces,
- lowering reductions generically.

Gluon shifts the boundary. The programmer controls more structure, but the compiler still handles complex mechanical lowering.

Misconception 5: Multi-Vendor Means Fully Portable Kernels

Gluon can target multiple vendors, but highly optimized kernels are not necessarily portable. A kernel using Blackwell-specific TCGEN05 instructions cannot run unchanged on Ampere.

A more accurate statement is:

Gluon is multi-target $\neq$ every Gluon kernel is portable.

Misconception 6: Warp Specialization Means Individual Thread Programming

Gluon remains at a coarse tensor and warp-group level. It generally does not require the programmer to manually elect a single thread or write per-lane control logic. For instructions that are issued by one lane or one warp, the lowering handles the required hardware convention.

Practical Takeaways

Takeaway 1: Layout Is a Performance Contract

In Gluon, a layout is not merely metadata. It is a contract between the programmer, compiler, and hardware:

layout $\Rightarrow$ ownership $\Rightarrow$ communication $\Rightarrow$ lowering $\Rightarrow$ performance.

Takeaway 2: Linear Layouts Turn GPU Indexing Into Algebra

The lecture's most important mathematical insight is:

GPU layout conversion $\approx$ matrix algebra over $\mathbb{F}_2$.

This unifies:

- register layouts,
- lane layouts,
- warp layouts,
- shared-memory layouts,
- swizzled layouts,
- tensor-core operand layouts,

- broadcasting and reduction ownership.

Takeaway 3: Blackwell Performance Requires Asynchrony

A high-performance Blackwell kernel is not a simple loop. It is a pipeline:

load next tile || compute current tile || store previous tile.

Gluon’s warp-specialized model lets the programmer express this pipeline directly.

Takeaway 4: Compiler Transformations Should Be Predictable

In Triton, the compiler may perform large transformations to infer warp specialization. In Gluon, the programmer writes the specialization explicitly, so profiling and performance reasoning are more direct.

Takeaway 5: Debugging Tools Are Essential

Because Gluon exposes more hardware details, correctness tools become important. The lecture mentions:

- concurrency sanitizer,
- invalid instruction sanitizer,
- floating-point sanitizer,
- global memory sanitizer,
- profiler instrumentation,
- layout visualizer.

These tools compensate for the additional complexity of lower-level programming.

Developer Tools

Proton Profiler

The lecture describes Proton as a profiling tool that supports Triton and Gluon. It can be used in several modes:

- coarse-grained profiling,
- fine-grained instrumentation,
- PC sampling.

A conceptual host-side profiling pattern is:

```

import proton

session = proton.start("matmul_profile")

proton.deactivate(session)
clear_l2_cache()
proton.activate(session)

matmul_kernel[grid](A, B, C, meta)

proton.finalize(session)

```

This allows uninteresting operations, such as cache clearing kernels, to be excluded from the profile.

Fine-Grained Instrumentation

Inside Gluon code, regions can be annotated:

```

def load_partition():
    profiler.enter_scope("load_partition")
    issue_tma_pipeline()
    profiler.exit_scope("load_partition")

def mma_partition():
    profiler.enter_scope("mma_partition")
    issue_mma_pipeline()
    profiler.exit_scope("mma_partition")

```

The resulting trace can show whether TMA, MMA, epilogue, and CLC partitions are balanced across time.

Concurrency Sanitizer

The concurrency sanitizer detects races such as:

- reading TMA-loaded data before waiting on the barrier,
- overwriting MMA inputs while the MMA engine still consumes them,
- incorrect producer-consumer synchronization across asynchronous stages.

A typical bug pattern is:

```

issue_tma_load(dst=smem[stage], barrier=barrier[stage])

# Bug: consumer reads before waiting.
tile = gl.load(smem[stage])

```

The correct pattern is:

```

issue_tma_load(dst=smem[stage], barrier=barrier[stage])

barrier.wait(stage)

tile = gl.load(smem[stage])

```

Invalid Instruction Sanitizer

This sanitizer checks hardware constraints such as TMA alignment requirements. TMA operations often require specific address alignment, shape constraints, and descriptor validity. Violating these constraints can cause invalid behavior that ordinary CUDA sanitizers may not diagnose clearly.

Floating-Point Sanitizer

The floating-point sanitizer replaces floating-point arithmetic with integer-like operations that preserve mathematical identities such as associativity. This is useful for testing compiler or algorithmic transformations that change rounding order.

The problem is that ordinary floating-point addition is not associative:

$$(a + b) + c \neq a + (b + c)$$

in general. Therefore, comparing optimized and unoptimized kernels with a small tolerance may hide structural errors. The sanitizer provides a more exact testing mode for transformations.

Global Memory Sanitizer

The global memory sanitizer detects races between CTAs and across kernels. It can support NVLink communication and cross-kernel synchronization patterns. The lecture notes that tensors must be allocated through a sanitizer-aware memory pool for global race checking.

Layout Visualizer

The layout visualizer helps users inspect how logical tensor elements map to hardware coordinates. It can show:

- preset layouts,
- layout matrices,
- register/thread/warp mappings,
- slicing through hardware dimensions,
- swizzled shared-memory layouts,

- results of layout operations such as compose and invert,
- PTX-documented instruction layouts such as MMA and LD matrix layouts.

This is important because linear layouts are mathematically compact but visually non-obvious.

Visual Concept

Draw a two-panel layout visualizer. The left panel shows hardware coordinates labeled by register, thread, and warp. The right panel shows logical tensor coordinates. Selecting one cell highlights the corresponding owner and logical element.

Project or Experiment Ideas

Experiment 1: Implement Naive Versus Tiled Matmul

Implement three matmul versions:

1. naive global-memory matmul,
2. shared-memory tiled matmul,
3. tensor-core tiled matmul using a DSL such as Triton or Gluon.

Measure:

$$\text{TFLOP/s} = \frac{2MNK}{t \cdot 10^{12}}.$$

Compare achieved throughput against theoretical peak.

Experiment 2: Study Layout Effects on Reduction

Create reductions where the reduced dimension is:

- within one thread,
- within one warp,
- across multiple warps,
- across multiple CTAs.

Measure how performance changes as the communication mechanism changes from registers to shuffles to shared memory to global memory.

Experiment 3: Visualize Linear Layouts

Write a small Python script that represents a layout matrix over $\mathbb{F}_2$. Given input bits, compute output bits:

```
import numpy as np

def f2_matmul(A, x):
    return (A @ x) % 2

A = np.array([
    [1, 0, 0, 1],
    [0, 1, 0, 0],
    [0, 0, 1, 1],
], dtype=np.uint8)

x = np.array([1, 0, 1, 1], dtype=np.uint8)
y = f2_matmul(A, x)

print(y)
```

Then interpret rows with multiple 1 entries as XOR-based swizzling.

Experiment 4: Bank Conflict Swizzle

Create a shared-memory transpose kernel with and without swizzling. Measure:

- shared-memory load/store throughput,
- bank conflict metrics in Nsight Compute,
- total kernel runtime.

Compare the address functions:

$$a_{\text{plain}}(i, j) = iN + j,$$

$$a_{\text{swizzled}}(i, j) = iN + (j \oplus f(i)).$$

Experiment 5: Pipeline Stage Count

Benchmark a tiled matmul with different stage counts:

$$S \in \{1, 2, 3, 4, 5\}.$$

Measure how runtime changes. More stages can improve latency hiding but increase shared-memory usage:

$$\text{SMEM usage} = S \cdot s(B_M B_K + B_K B_N).$$

Too many stages may reduce occupancy.

Experiment 6: TMA Multicast Effect

For a cluster-level matmul, compare:

- independent TMA loads per CTA,
- multicast TMA load shared across CTAs.

Expected improvement appears when multiple CTAs reuse the same operand tile.

Final Mental Model

Gluon is best understood as a DSL for programmers who want to explicitly describe the structure of a modern GPU kernel without dropping all the way to raw PTX.

Triton asks the compiler to discover the hardware schedule. Gluon asks the programmer to expose the schedule, while the compiler handles the algebraic and mechanical lowering.

The central abstraction is the layout. A layout tells you where tensor elements live:

logical tensor $\longleftrightarrow$ registers, lanes, warps, shared memory, tensor memory.

Linear layouts make this precise by representing layouts as binary matrices:

$$y = Ax \quad \text{over } \mathbb{F}_2.$$

This turns indexing, swizzling, broadcasting, reduction ownership, and layout conversion into linear algebra. That is the mathematical backbone of Gluon.

For Blackwell performance, the key kernel mental model is:

TMA producer || MMA consumer || epilogue worker || scheduler.

A speed-of-light kernel is not merely a tiled loop. It is a carefully synchronized asynchronous pipeline whose data movement, computation, and output conversion are all overlapped. Gluon gives the programmer control over this pipeline while preserving enough abstraction to avoid writing every register, lane, bank, and barrier operation by hand.